Claudio Pelleriti
LM Ingegneria Informatica
Visione Artificiale

Real vs AI-Generated Faces Classification with Deep Learning

3° ASSIGNMENT

UNIVERSITÀ DEGLI STUDI DI PALERMO
dipartimento di ingegneria unipa

Introduzione

Le immagini di volti generate da AI sono sempre più realistiche.
Riconoscere un volto reale da uno generato è cruciale per la sicurezza e la fiducia nei contenuti digitali.
L'obiettivo di questo assignment è quindi quello di costruire un classificatore binario per distinguere volti reali da volti generati da AI.

REAL FAKE

Dataset Costruction

Dataset da 100.000 immagini

Per addestrare i modelli ho costruito due dataset, uno più piccolo, da 20.000 immagini totali, bilanciate tra le due classi real e fake, e uno da 80.000 immagini totali. Per la classe di immagini real ho usato immagini prese dal dataset: FFHQ (Flickr-Faces-HQ), mentre per le immagini fake: AI-Generated Faces - Kaggle

Dataset da 20.000 immagini

Ho usato il dataset più piccolo, con cui potevo fare addestramenti più veloci, per fare diverse prove e scegliere l'architettura migliore del modello: quali layer usare, come fare fine tuning, e dove posizionare il layer di attention.

Dataset da 80.000 immagini

Una volta scelta l'architettura migliore e salvato il modello addestrato con il dataset da 20.000 immagini, ho utilizzato il dataset da 80.000 immagini per continuare ad addestrare i modelli con e senza attention in modo da provare a ottenere (in tempi più lunghi) prestazioni ancora migliori.

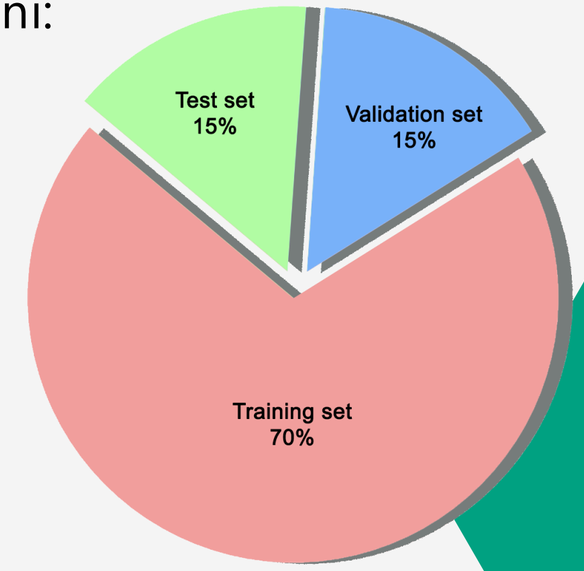
Dataset Organization and Experimental Protocol

Il dataset costruito era già perfettamente bilanciato tra le due classi. Per evitare overfitting e aumentare la robustezza del modello ho fatto data agumentation con i seguenti parametri randomicamente apportati alle immagini:

- Flip orizzontale
- Rotazione ±10°
- Zoom ±10%
- Variazione di contrasto ±10%

Le immagini vengono mescolate casualmente e suddivise in:

- **70%** → Training set
- **15%** → Validation set
- **15%** → Test set



Architettura dei modelli

Ho sperimentato due diverse archietture, che differiscono sostanzialmente per l’ uso come base per la feature extraction di due reti preaddestrate diverse:

Architettura dei modelli

Ho sperimentato due diverse archietture, che differiscono sostanzialmente per l' uso come base per la feature extraction di due reti preaddestrate diverse:

1° architettura: ResNet50

- Base: **ResNet50** pre-addestrata su ImageNet
- Fine-tuning: solo gli ultimi 10 layer trainable
- Pooling globale + Dense(128) (testata sia con che senza dropout)
- modulo di Attention prima del classificatore (per il modello con attention)
- Output: softmax su 2 classi

Architettura dei modelli

Ho sperimentato due diverse archietture, che differiscono sostanzialmente per l' uso come base per la feature extraction di due reti preaddestrate diverse:

1° architettura: ResNet50

- Base: **ResNet50** pre-addestrata su ImageNet
- Fine-tuning: solo gli ultimi 10 layer trainable
- Pooling globale + Dense(128) (testata sia con che senza dropout)
- modulo di Attention prima del classificatore (per il modello con attention)
- Output: softmax su 2 classi

2° architettura: EfficientNetB0

- Base: **EfficientNetB0**, più leggera ed efficiente, anch'essa pre-addestrata su ImageNet
- Fine-tuning: solo gli ultimi 10 layer trainable
- Pooling globale + Dense(128) + BatchNormalization + Dropout(0.5)
- modulo di Attention prima del classificatore (per il modello con attention)
- Output: softmax su 2 classi

Training

In entrambi i casi il training il training aveva le seguenti caratteristiche:

- Ottimizzatore: Adam
- Loss: sparse categorical crossentropy
- Epoche: massimo 30
- Batch size: 32

e le seguenti callbacks:

- Earlystopping con monitoraggio della loss sul validation set, patience = 5 (più alta per i modelli il cui andamento era più irregolare), e il ripristino dei pesi migliori
- ReduceLROnPlateau con monitoraggio della loss su validation set, patience = 3 (più alta per i modelli il cui andamento era più irregolare), fattore di riduzione 0.5 e learning rate minimo 10^{-6}

Model selection

La seconda architettura è risultata essere molto più efficiente e performante, in particolare quella con il layer di attention.

Di seguito mostro i risultati su validation set per I diversi modelli.

Model selection

La seconda architettura è risultata essere molto più efficiente e performante, in particolare quella con il layer di attention.

Di seguito mostro i risultati su validation set per I diversi modelli.

Con la **prima architettura**:

- Modello senza attention addestrato su dataset da 20K immagini
  - Loss: 0.5817
  - Accuracy: 0.7302

Model selection

La seconda architettura è risultata essere molto più efficiente e performante, in particolare quella con il layer di attention.

Di seguito mostro i risultati su validation set per I diversi modelli.

Con la **prima architettura**:

- Modello senza attention addestrato su dataset da 20K immagini
  - Loss: 0.5817
  - Accuracy: 0.7302

- Modello senza attention addestrato su dataset da 100K imagini
  - Loss: 1.4069
  - Accuracy: 0.6266

Model selection

La seconda architettura è risultata essere molto più efficiente e performante, in particolare quella con il layer di attention.

Di seguito mostro i risultati su validation set per I diversi modelli.

Con la **prima architettura**:

- Modello senza attention addestrato su dataset da 20K immagini
  - Loss: 0.5817
  - Accuracy: 0.7302

- Modello senza attention addestrato su dataset da 100K imagini
  - Loss: 1.4069
  - Accuracy: 0.6266

Con la **seconda architettura**:

- Modello senza attention addestrato su dataset da 20K immagini
  - Loss: 0.0381
  - Accuracy: 0.9832

Model selection

La seconda architettura è risultata essere molto più efficiente e performante, in particolare quella con il layer di attention.

Di seguito mostro i risultati su validation set per I diversi modelli.

Con la **prima architettura**:

- Modello senza attention addestrato su dataset da 20K immagini
  - Loss: 0.5817
  - Accuracy: 0.7302

- Modello senza attention addestrato su dataset da 100K imagini
  - Loss: 1.4069
  - Accuracy: 0.6266

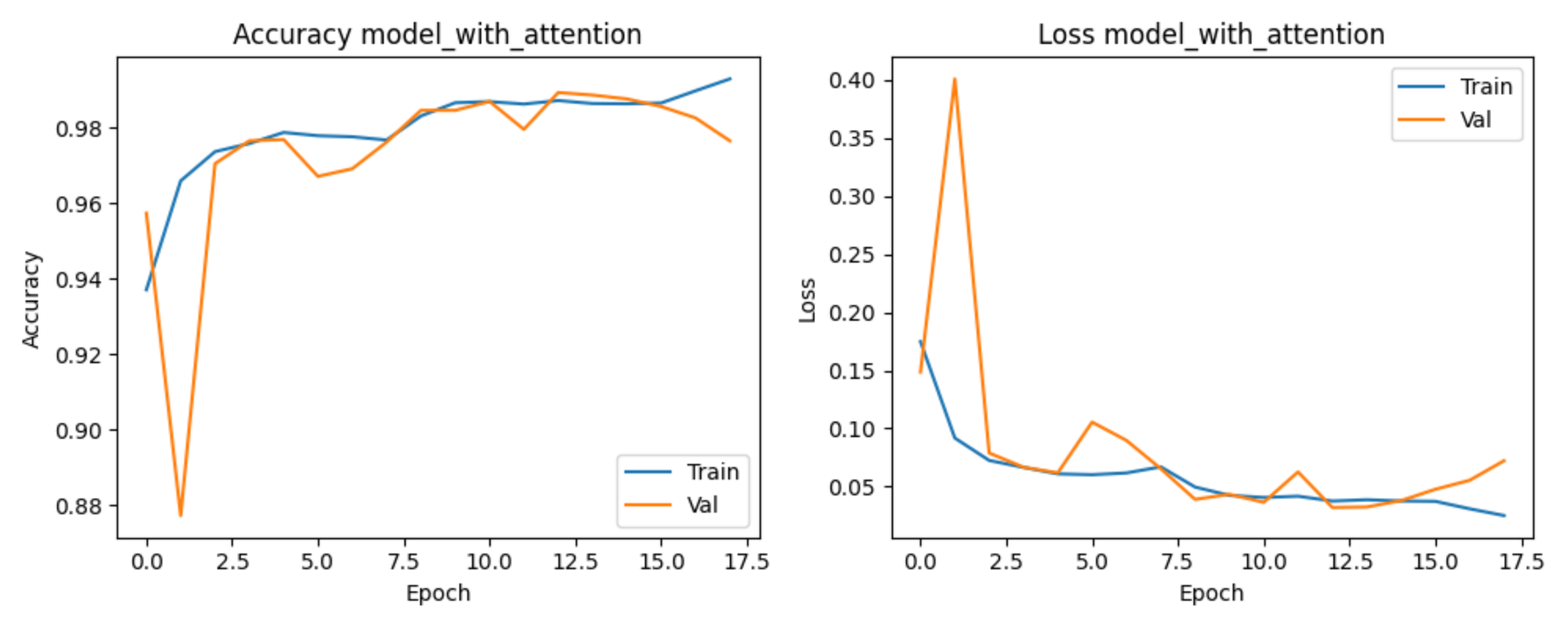
Con la **seconda architettura**:

- Modello senza attention addestrato su dataset da 20K immagini
  - Loss: 0.0381
  - Accuracy: 0.9832

- **Modello con attention addestrato su dataset da 20K immagini**
  - **Loss: 0.0301**
  - **Accuracy: 0.9899**

Modello migliore

Grafici accuratezza e loss del modello migliore



Commenti sui risultati ottenuti

Risultati migliori con EfficientNetB0 piuttosto che ResNet50
Dalle ricerche EfficientNet risutla infatti avere rispetto alla ResnNet50

- **Architettura più efficiente**
  - Compound Scaling: bilancia profondità, larghezza e risoluzione.
  - Migliori prestazioni con meno parametri (~5.3M invece che ~25M).
- **Migliore generalizzazione**
  - Struttura moderna e regolarizzata.
  - Più stabile e meno incline all'overfitting.
- **Ottimizzazione hardware**
  - Progettata per sfruttare al meglio GPU/TPU.
  - Eccellente compatibilità con chip Apple M1 (Presente nella macchina che ho usato per l'addestramento).
- **Feature extraction più potente**
  - Riconosce dettagli visivi più fini.
  - Ideale per l'obiettivo di questo assignment.
- **Pretraining solido**
  - Pre-addestrata su ImageNet-1K con tecniche moderne.
  - Base più robusta per il fine-tuning.

Commenti sui risultati ottenuti

Il modello con il **layer di Attention** ha ottenuto prestazioni leggermente migliori del modello senza attention, ma comunque molto simili (meno di un punto percentuale)

Scelgo comunque come modello migliore quello con il Layer di Attention per avere una migliore **Explainability**, il layer di Attention infatti, nonostante non sia un'attention spaziale (che permetterebbe di individuare il punto sull'immagine in cui si trovano le feature più o meno significative), permette comunque di visualizzare quali feature vengono pesate di più, aiutando a giustificare meglio le decisioni del modello, rendendolo quindi un modello più trasparente.

Reference

Building and training model from Resnet 50:
https://wandb.ai/mostafaibrahim17/ml-articles/reports/The-Basics-of-ResNet50---Vmlldzo2NDkwNDE2#step-3:-load-and-preprocess-the-dataset

EfficientNetB0:
https://blog.tensorflow.org/2020/03/higher-accuracy-on-vision-models-with-efficientnet-lite.html
https://paperswithcode.com/paper/efficientnetv2-smaller-models-and-faster

Salvare e caricare modello con keras
https://www.intelligenzaartificialeitalia.net/post/come-salvare-e-caricare-il-tuo-modello-di-deep-learning-o-apprendimento-profondo

Attention Layer:
Slide professoressa Liliana Lo Presti

EarlyStopping:
Slide prof Liliana Lo presti